Digital Products & UX / Service & Transformation

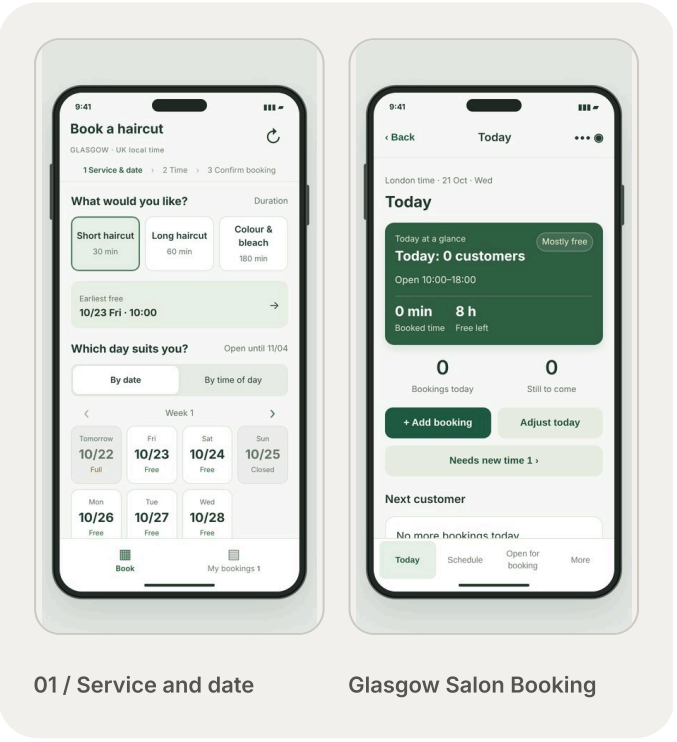
Glasgow Salon Booking

One shared diary. Two clear experiences.

A customer and shop booking system for a small Glasgow salon run by an older owner, who currently manages every appointment through WeChat, a paper diary and memory.

Interactive two-sided prototype · field research planned

Try the interactive prototype: **demos/salon-booking/**



01 / Service and date — Glasgow Salon Booking

MY CONTRIBUTION	Problem framing, service blueprint, booking rules, UI for both sides and AI-assisted prototyping
SCOPE	Customer booking + shop diary and availability
TYPE	Service design · booking system
STAGE	Working prototype, before owner testing
BUILT WITH	TypeScript booking core, AI-assisted within rules

The aim is not to put booking online. It is to let the routine run itself, so the owner only steps in for the exceptions — without learning a complicated tool.

01 / The problem

How might every customer book easily — without adding work for the owner?

Everything lives in one person

WeChat + paper diary + memory.

One long service delays everyone after it

Times were written down; durations were not.

Plans change — and every change means messaging everyone

Illness or travel = messaging everyone again.

Customers have to ask before they can book

No WeChat, no easy way in.

The owner

Runs the salon alone | WeChat + paper diary | Hours change with health and travel | Away for two weeks recently | Needs tools she can read at a glance

"Is anyone booked on Friday? Who do I need to tell?"

Same booking, different questions

Customer asks

What services? | Which days are open? | Which times are free? | What did I book?

One shared diary

Owner asks

How many today? | Who, when, what, how long? | Where is free time? | Who is affected if I close?

Three users, three ways in

Chinese customers

WeChat mini program · V1

Open → | Service → | Time → | Confirm

Non-WeChat customers

Owner adds them · own entry next

Call or walk in → | Owner adds booking → | Same diary

Owner

Shop view

Today → | Set hours once → | Handle exceptions

Where this stands

> Built from my background and pain-point notes on the salon, a booking-logic specification and local self-tests. A structured interview and on-site test with the owner come next; services, hours and bookings in the demo are fixtures.

Customer

Choose a service and time, understand the duration, then find and change the appointment.

Shop owner

See the day, enter a manual booking, adjust opening hours and resolve affected appointments.

Customers without WeChat

Considered from the start; in V1 the owner adds them, with their own entry next.

---

02 / How the system works

Service blueprint: who does what

Routine work moves to the system. The owner acts in only two places.

Customer

Choose · Picks a service

Find a time · By date or fixed time

Book · Confirms

Before visit · Confirmation + day-before reminder

Visit · Arrives

Exception · Rebooks, service kept

Owner

Choose · —

Find a time · Sets hours once

Book · Nothing

Before visit · Nothing

Visit · Just cuts hair

Exception · Closes a day once

System

Choose · Knows its duration

Find a time · Only times that fit

Book · Rechecks — no double booking

Before visit · Sends WeChat reminders

Visit · Status updates itself

Exception · Moves bookings to “reschedule”

When the owner suddenly can’t open

1. Customer books 13:00
2. Owner closes 13:00–14:00
3. Booking becomes “to reschedule”
4. Customer picks a new time, service kept
5. Owner’s diary updates

Tested end to end across both sides of the prototype.

03 / Design principles

Booking rules that protect the owner

01
The routine runs itself
Owner handles exceptions only.

02
Duration decides availability
150-min gap: yes to a cut, no to a colour.

03
Only valid start times
No “17:45” — the day stays unfragmented.

04
Recommend, don’t decide
Suggested times reduce effort; any valid time stays open.

05
“Not open yet” ≠ “full”
Owner sets how far ahead to open; no same-day bookings.

06
Protect long gaps
Fill small gaps; keep big blocks free.

07

08

Never delete a booking

Closures move bookings to rescheduling.

Low learning cost

Starts from “today”, like her paper diary.

Deliberately not in V1

~~Online payment~~ ~~Coupons & points~~ ~~Multiple shops~~ ~~Parallel services~~ ~~No-show penalties~~ ~~AI chat~~

---

04 / Process & iteration

How I worked

01 Pain points → 02 V1 scope → 03 Three user flows → 04 Rules + AI guardrails → 05 Build & test

---

Booking rules, data contract and what the AI may change were fixed before any code was written.

Customer flow

BEFORE
One long schedule list on the home screen

→

AFTER
Three steps: week overview → valid times → confirm

Detailed diary folded away; Back keeps the selection.

Owner’s side

BEFORE
About 10 scattered settings pages

→

AFTER
4 time-first tabs: Today · Diary · Availability · More

“No-show” moved under more actions, so the routine stays automatic.

Two developers, one core

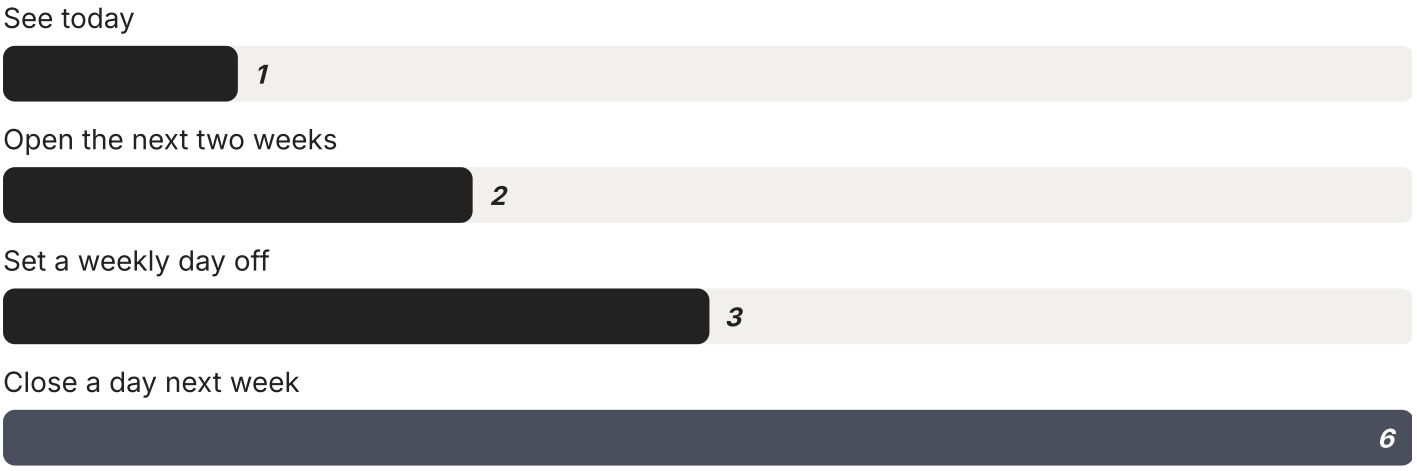
BEFORE
Customer and owner sides built separately

→

AFTER
One shared booking core, changes agreed first

e.g. how the owner adds a non-WeChat customer was raised as a change request.

Owner tasks, measured in taps



Target ≤ 4 taps

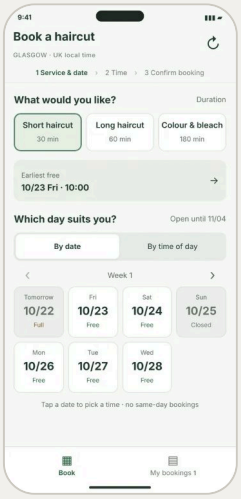
Target: 3–4 taps. Closing a future day still takes 6 — the next thing to simplify.

> **Evidence**
> Design reviews and 82 automated tests, including the full cross-side closure scenario. Not yet tested with the owner herself.

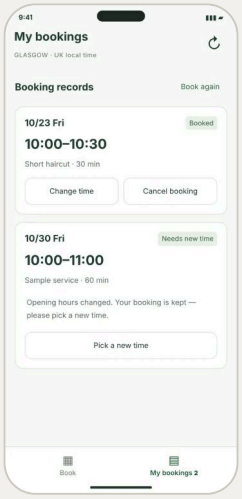
05 / Screen map

Captured from the public prototype (English layer on). Level 1 is the bottom navigation; each row follows one path deeper. Select any screen to enlarge it.

L1 Tab bar



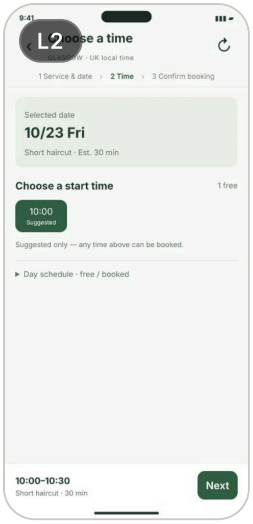
Book



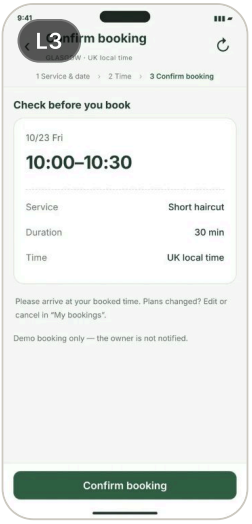
My bookings

Key flow Book → three steps

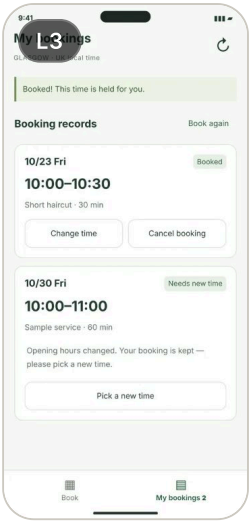
01 02 03



Choose a time

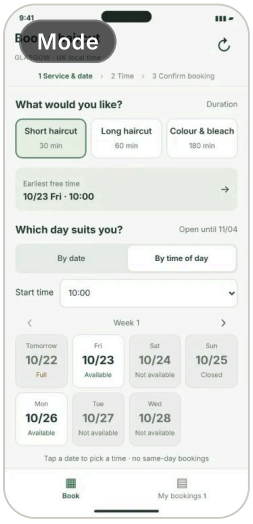


Check before booking



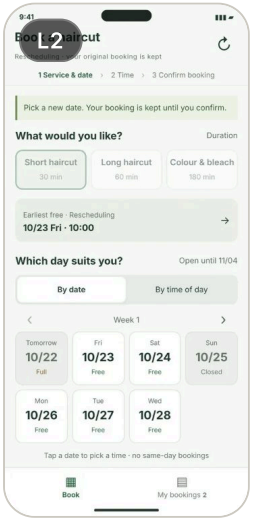
Booked

Book → search by time of day

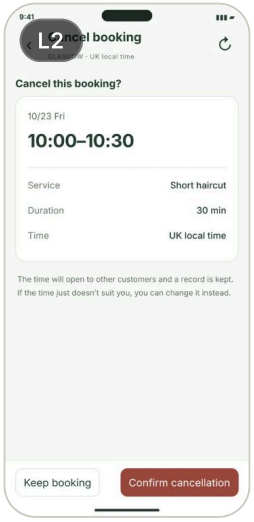


By time of day

My bookings → change of plan

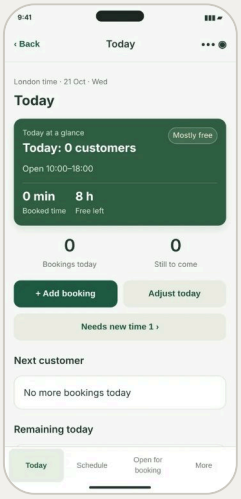


Change time

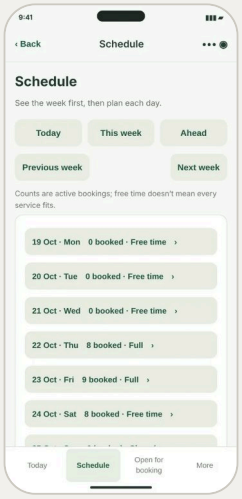


Cancel

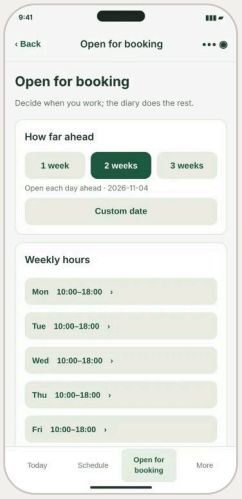
L1 Tab bar



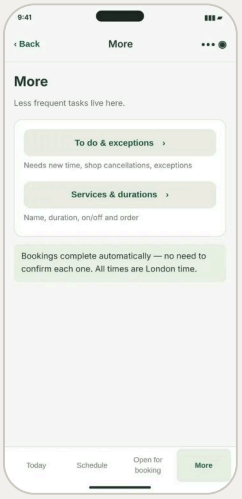
Today



Schedule



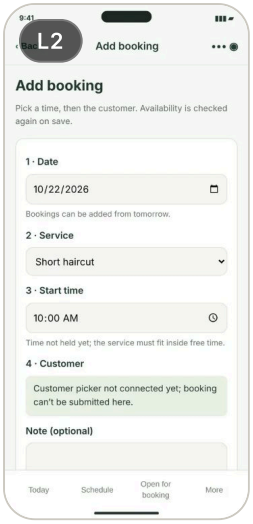
Open for booking



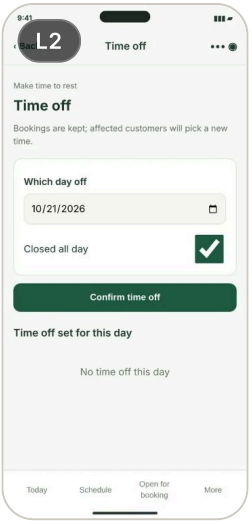
More

Key flow Today → the owner's routine

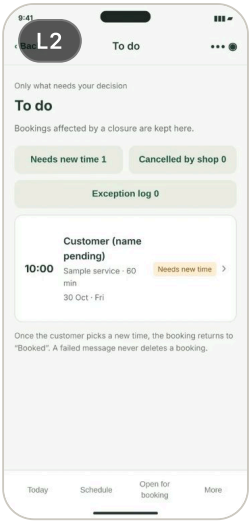
01 02 03



Add a booking

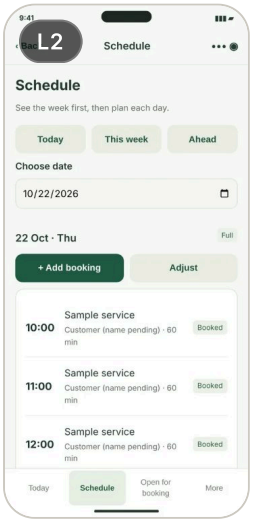


Take time off

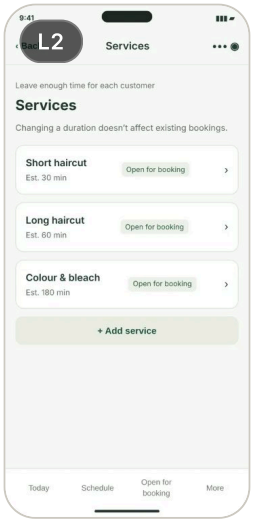


Needs a new time

Schedule & More



A day in the diary



Services & durations

---

06 / The end-to-end journey

01

Choose

Select a service and available date.

02

Check

See starts that fit the full duration.

03

Confirm

Review service, duration and local time.

04

Manage

Find the same appointment in My bookings and the shop diary.

05

Recover

If hours change, retain the booking and arrange a new time.

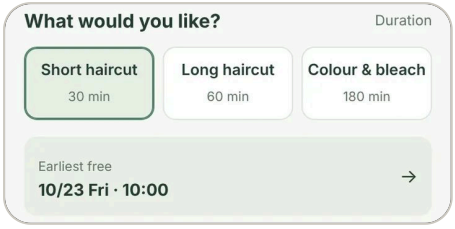
Back preserves a valid selection. Changing the service or date clears an incompatible time. Rescheduling reuses

the selection flow; cancellation requires its own confirmation.

---

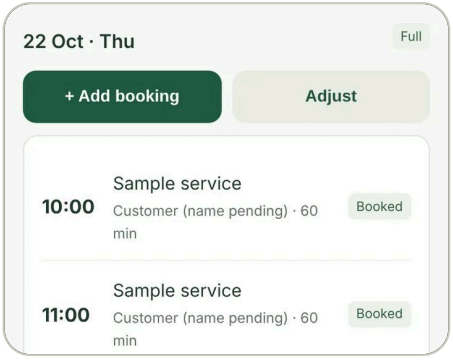
07 / Design details

Close-ups of the decisions that carry the product. Select any image to enlarge it.



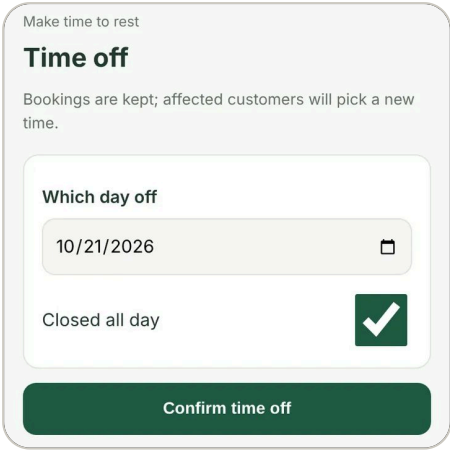
Duration is chosen first

Each service shows its length, and the earliest free time updates with it, because duration decides what fits.



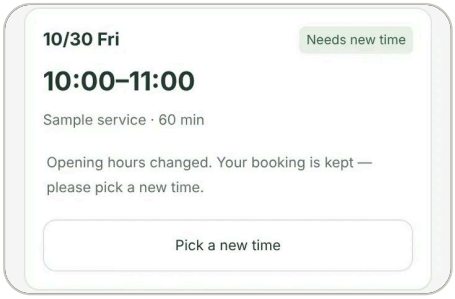
The diary, not the chat

The owner sees a day as times, services and status, with Add and Adjust one tap away.



Time off never deletes a booking

The rule is written on the screen: bookings are kept and affected customers pick a new time.



Customers see why, and what to do

A closure turns the booking into “Needs new time” with a one-tap way to choose again.

---

08 / How it was built

The logic was fixed before any screen: pain points, V1 scope, three user flows, then technical rules for the AI tools. Visual polish came last.

01

One booking core

Availability, booking, rescheduling, closures and admin actions live in one shared TypeScript core used by both sides.

02

Guardrails for AI

A written spec, folder rules and a list of what the AI may and may not change kept generated code inside the design.

03

Two front ends

A customer flow and a shop diary, both phone-sized; the target is a WeChat mini program with a separate entry for non-WeChat customers.

04

Public prototype

The real booking rules are bundled to run in the browser, so anyone can try both sides without a server.

82

automated tests

4

tabs for the owner

3

steps for a customer to book

> **Limits**
>
> Tests check the rules; they do not show that the owner finds it easy. That needs the interview and on-site test below.

---

09 / Challenges, gaps & opportunities

What is hard

A low-confidence user

The owner is used to WeChat and a paper diary. Every extra step is a reason to go back to them, so the shop side had to shrink to four tabs.

Durations are uncertain

Hair length and colour change how long a service takes, so estimates can still overrun.

Two kinds of customer

WeChat suits Chinese customers but not everyone else. V1 serves WeChat; others are added by the owner until their own entry is ready.

Not done yet

- Cloud storage and real WeChat sign-in
- Reliable notifications after a closure
- Interview and on-site test with the owner
- Own entry for non-WeChat customers

Opportunities

Fewer messages

If customers can see free times themselves, most “are you open?” messages disappear.

A wider local audience

A non-WeChat entry opens the shop to local customers who would never add a private WeChat.

A pattern for other small shops

The same core could serve other one-person services with changing hours.

How success would be measured

- ◎ Owner tasks in four taps or fewer
- ◎ Share of bookings made without chatting
- ◎ Time spent rescheduling after a closure
- ◎ Owner still using it after a month

Proposed measures for a pilot, not results.

10 / Validation plan

Planned fieldwork · no findings claimed yet

The next phase connects the prototype to real people and settings. The following is a research plan, not a report of completed activities.

01

Observe a real booking

With permission, map how a request moves from a conversation to the diary, including changes and interruptions. Record steps without retaining private customer details.

02

Validate the operating rules

Confirm service lengths, opening windows, same-day policy, buffers and preferred language with the owner. Demo fixtures are not final business rules.

03

Test both sides of a change

Ask a customer to book and reschedule, then ask the owner to adjust a period and identify affected appointments. Record completion, errors, time and assistance.

11 / Progress & release plan

The public prototype runs the project's real booking rules in the browser, with sample data stored only on the visitor's device. The UI was written in Chinese for WeChat customers; an English layer is on by default. CloudBase persistence, real WeChat identity, reliable notifications and production deployment are unfinished. No real customer is charged or notified.

> **Now · customer and shop prototype, public and runnable in the browser**
>
> Next · owner interview, observation and two-sided task testing
>
> Then · refine service rules and run an agreed limited pilot
>
> Before release · persistent data, identity, notification delivery and recovery checks

Local implementation checks do not establish usability or business impact. Release and a real pilot require further validation and explicit approval.